Cairo University

Faculty of Computers and Artificial Intelligence

CS251

Introduction to Software Engineering

Personal Budgeting

Software Design Specifications

Version 1.0

ID	Name	Email	Mobile
20230228	Abdallah Kareem Abdallah	20230228@stud.fci-cu.edu.eg	01157803427
20230454	Hady Hassan Ahmed	20230454@stud.fci-cu.edu.eg	---
20230143	Reyad Youssef Ebrahim	20230143@stud.fci-cu.edu.eg	---

April of 2025

CS251: The Originals
Project: Personal Budgeting

Software Design Specification

Contents

Team 3
Document Purpose and Audience 3
System Models 3
I. Architecture Diagram 3
II. Class Diagram(s) 4
III. Class Descriptions 5
IV. Sequence diagrams 6
Class - Sequence Usage Table 7
V. State Diagram 8
VI. SOLID Principles 8
VII. Design Patterns 8
Tools 8
Ownership Report 8

CS251: The Originals
Project: Personal Budgeting

Software Design Specification

Team

ID	Name	Email	Mobile
20230228	Abdallah Kareem Abdallah	20230228@stud.fci-cu.edu.eg	01157803427
20230454	Hady Hassan Ahmed	20230454@stud.fci-cu.edu.eg	01125449653
20230143	Reyad Youssef Ebrahim	20230143@stud.fci-cu.edu.eg	01275525943

Document Purpose and Audience

Purpose

The SDS provides a clear, structured blueprint for building the software. It translates user needs into detailed technical designs, ensuring all team members understand how the system should work. This helps guide development, improves collaboration, supports testing, and makes future updates easier.

Audience

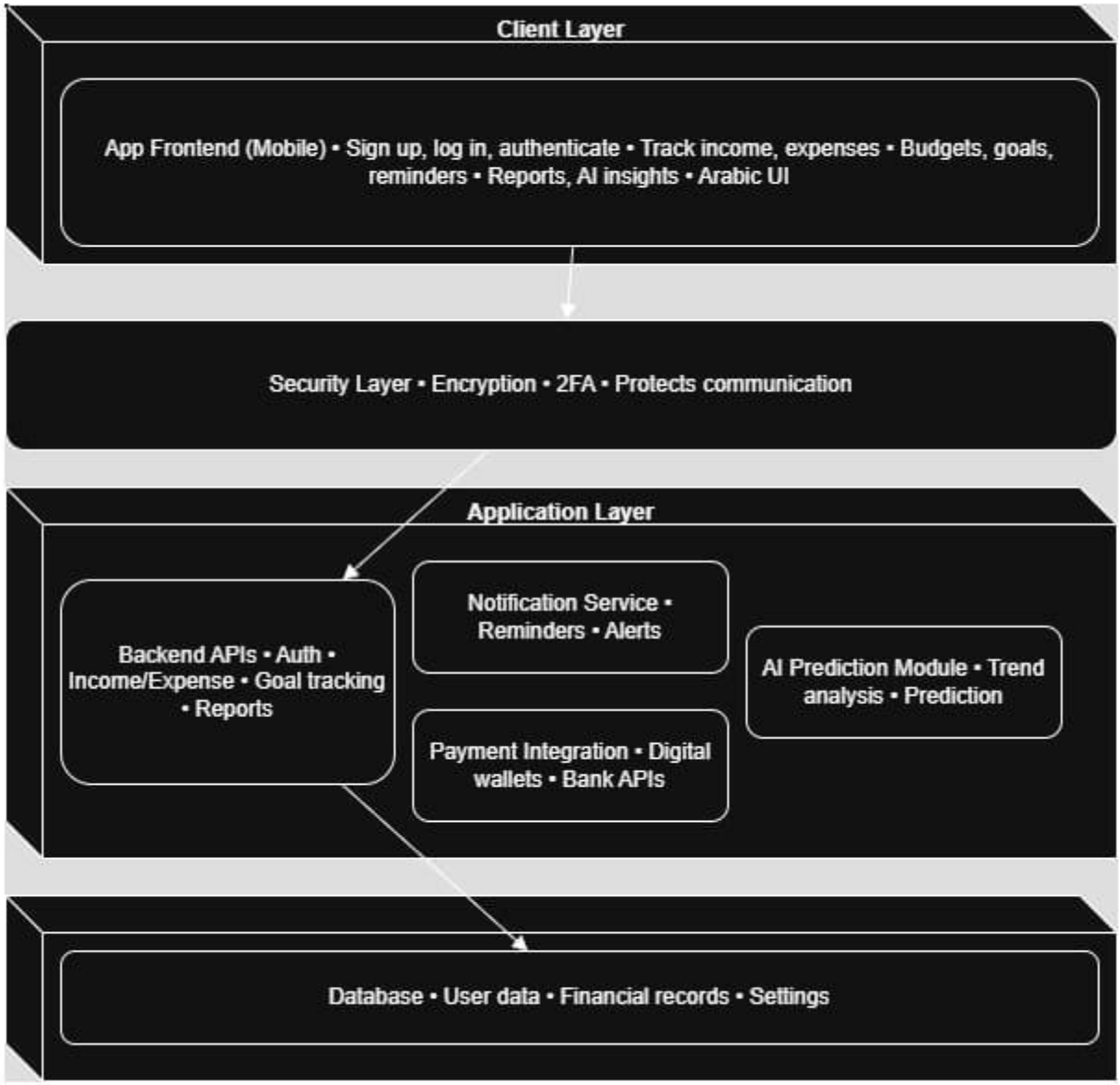
- **Software Developers** – To understand how to build the system and implement features.
- **Project Managers** – To track design progress and ensure alignment with project goals.
- **Testers/QA Engineers** – To design test cases based on expected functionality and design logic
- **Stakeholders/Clients (if technical)** – To review and approve the proposed design before implementation.
- **Maintenance Engineers** – To troubleshoot or upgrade the system later on

CS251: The Originals
Project: Personal Budgeting

Software Design Specification

System Models

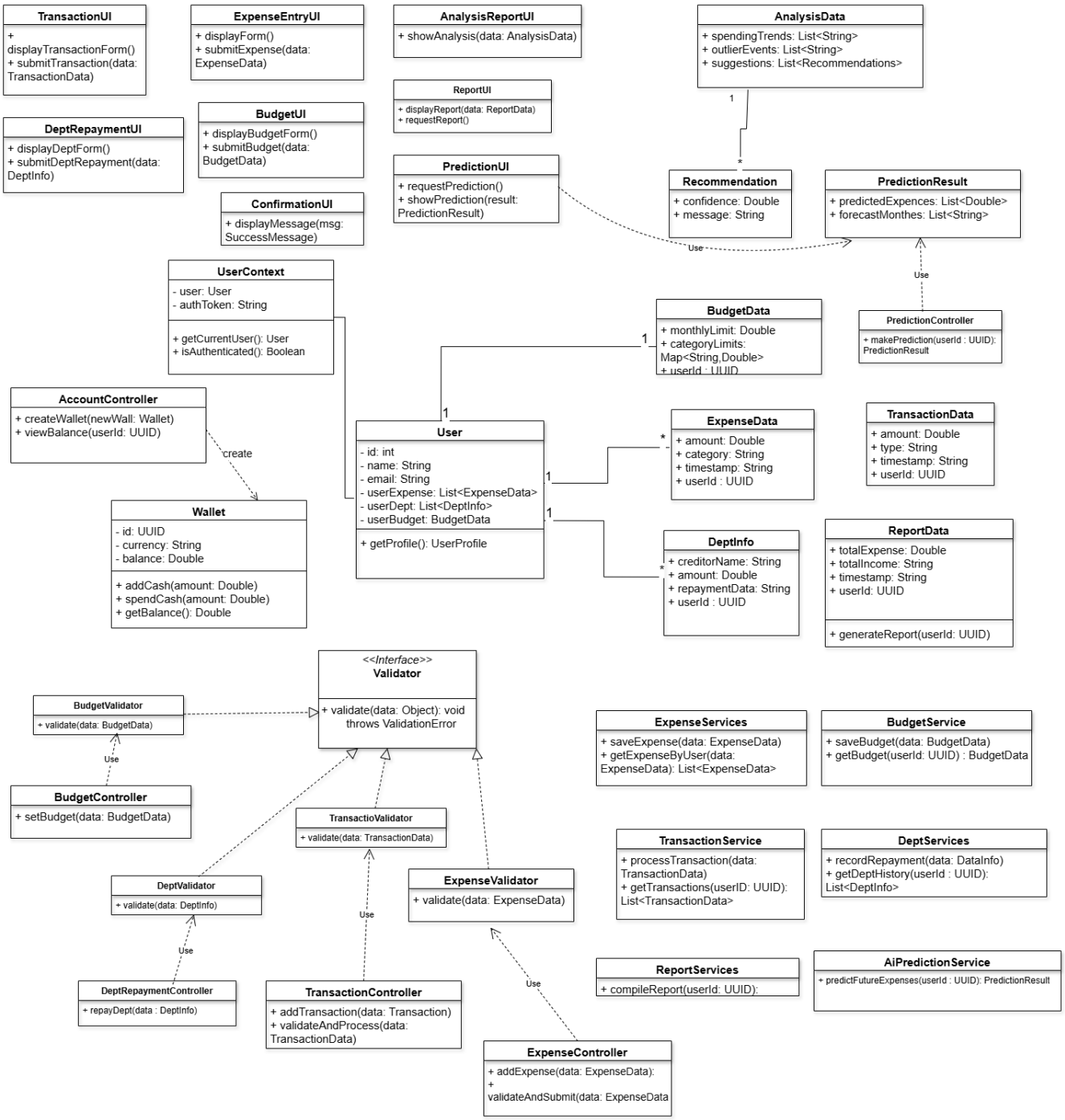
I. Architecture Diagram



CS251: The Originals
Project: Personal Budgeting

Software Design Specification

II. Class Diagram(s)



Software Design Specification

III. Class Descriptions

Class ID	Class Name	Description & Responsibility
C01	User	• Manage user authentication and personal information. • Represents a system user with login credentials and preferences. Handles user login, password management, and biometric authentication.
C02	DeptInfo	• Track and manage debts. • Keeps information about user's debts including the name, total amount, and remaining balance. Provides functions to add debt and track repayment.
C03	Expense	• Track and manage user expenses • Stores details of expenses such as category, amount, and payment method. Supports adding, editing, and deleting expenses.
C04	Income	• Record and manage income entries • Holds information about sources of income including whether they are recurring. Provides functions to add, edit, or delete income records.
C05	Transaction	• Handle financial transactions. • Represents a transaction with its details like category, type as income or expense, and date. Can be scheduled or cancelled if recurring.
C06	Reminder	• Set and manage reminders • Allows creation of reminders with specific titles, dates, and times. Can also send notifications.
C07	BudgetData	• Manage user's budget plan. • Tracks overall budget and limits per category for a certain duration. Allows setting, editing, comparing, and generating recommendations • Contains the user's budget details including monthly limit and categorized budget allocations.
C09	Wallet	• Manages a user's virtual wallet including balance operations. Allows adding and spending cash and we can connect this wallet with any bank
C10	ReportData	• Summarizes a user's financial report, including total



Software Design Specification

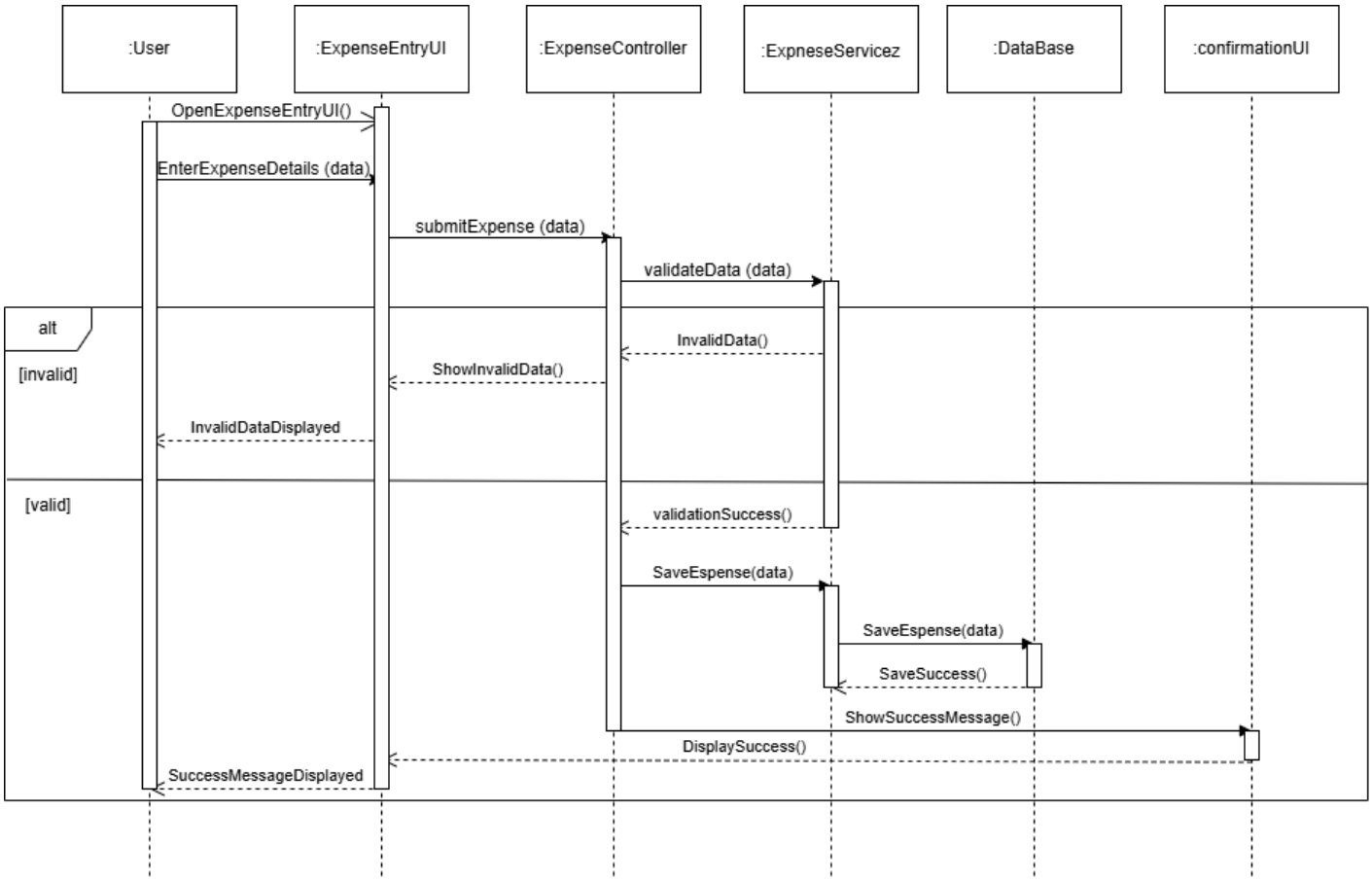
		expenses and income over a time period.
C11	PredictionResult	• Stores the result from AI predictions such as predicted expenses and forecasted months
C12	Recommendation	• Contains financial suggestions and their confidence levels to help users improve budgeting
C13	UserContext	• Manages session-related user info like current user and authentication token

CS251: The Originals
Project: Personal Budgeting

Software Design Specification

IV. Sequence diagrams

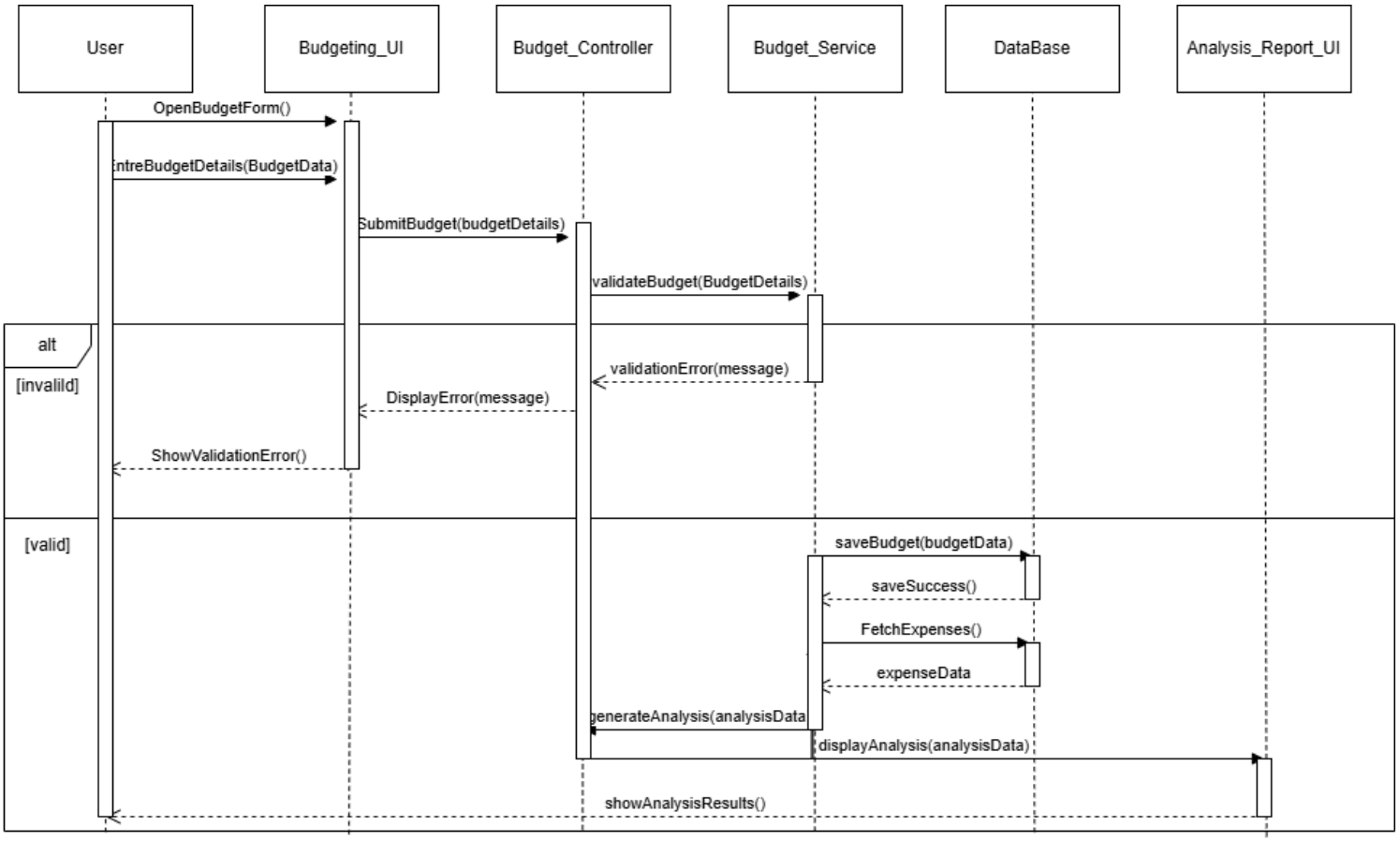
Sequence diagram 1



CS251: The Originals
Project: Personal Budgeting

Software Design Specification

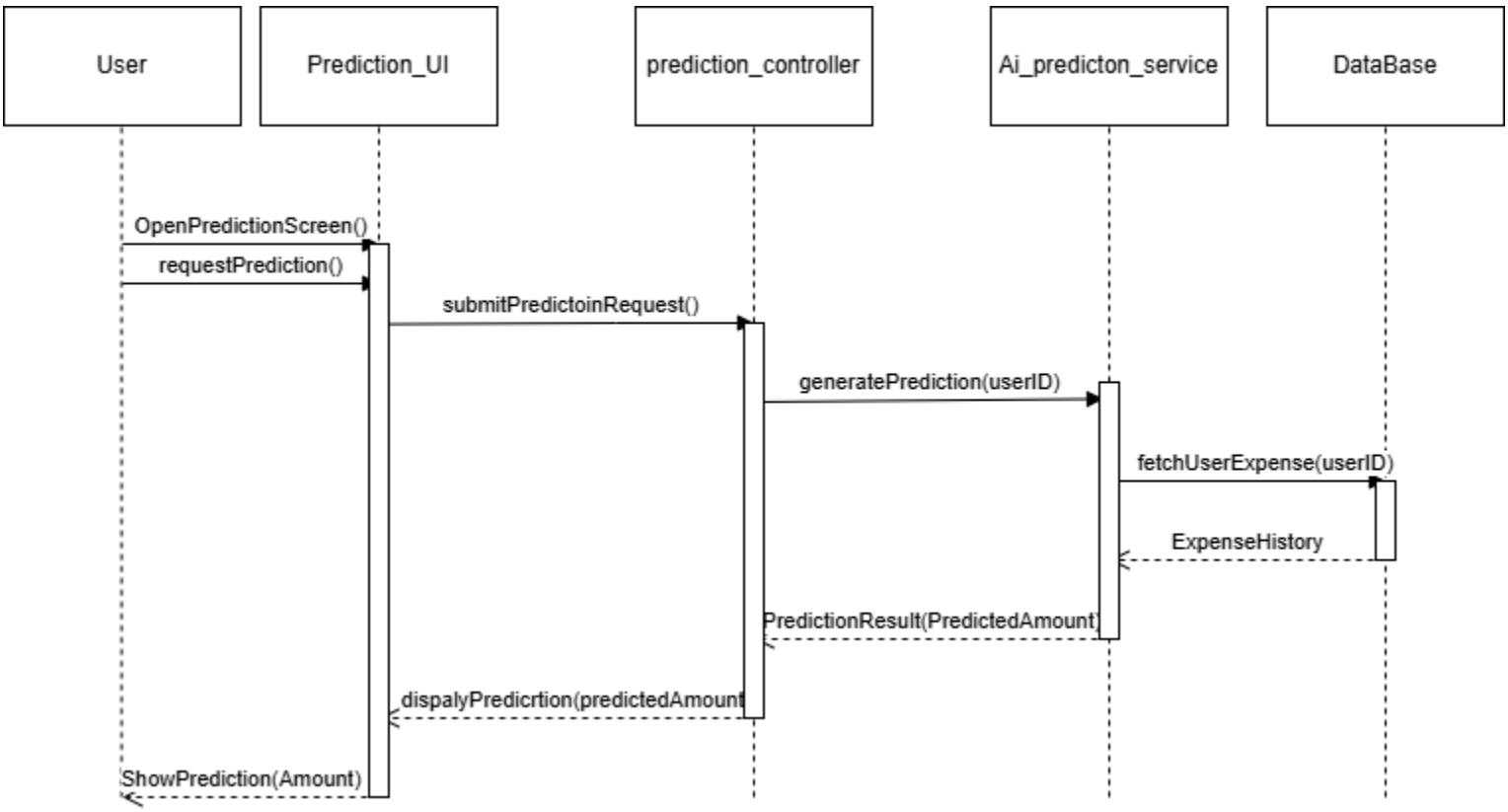
Sequence Diagram 2



CS251: The Originals
Project: Personal Budgeting

Software Design Specification

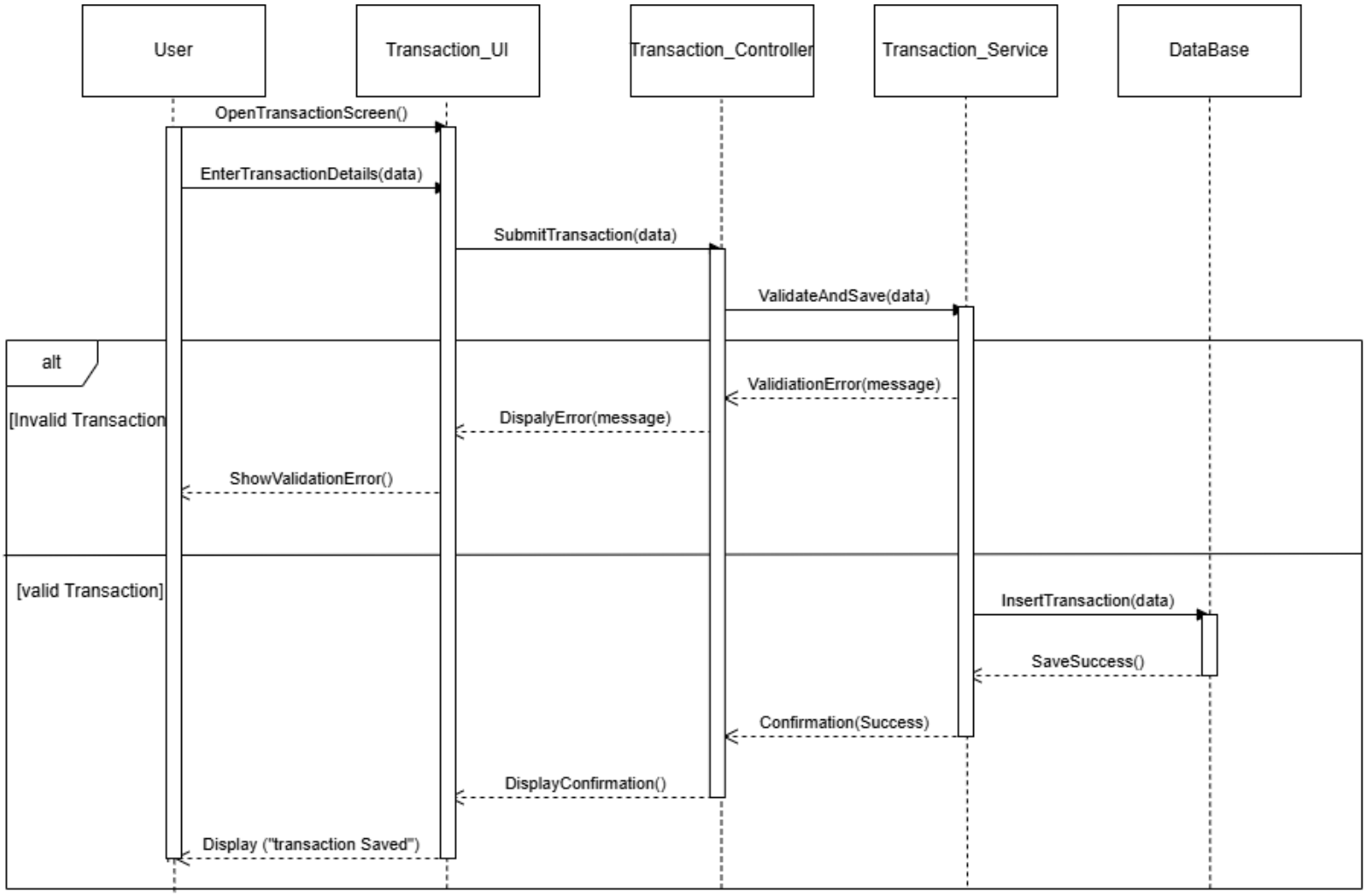
Sequence diagram 3



CS251: The Originals
Project: Personal Budgeting

Software Design Specification

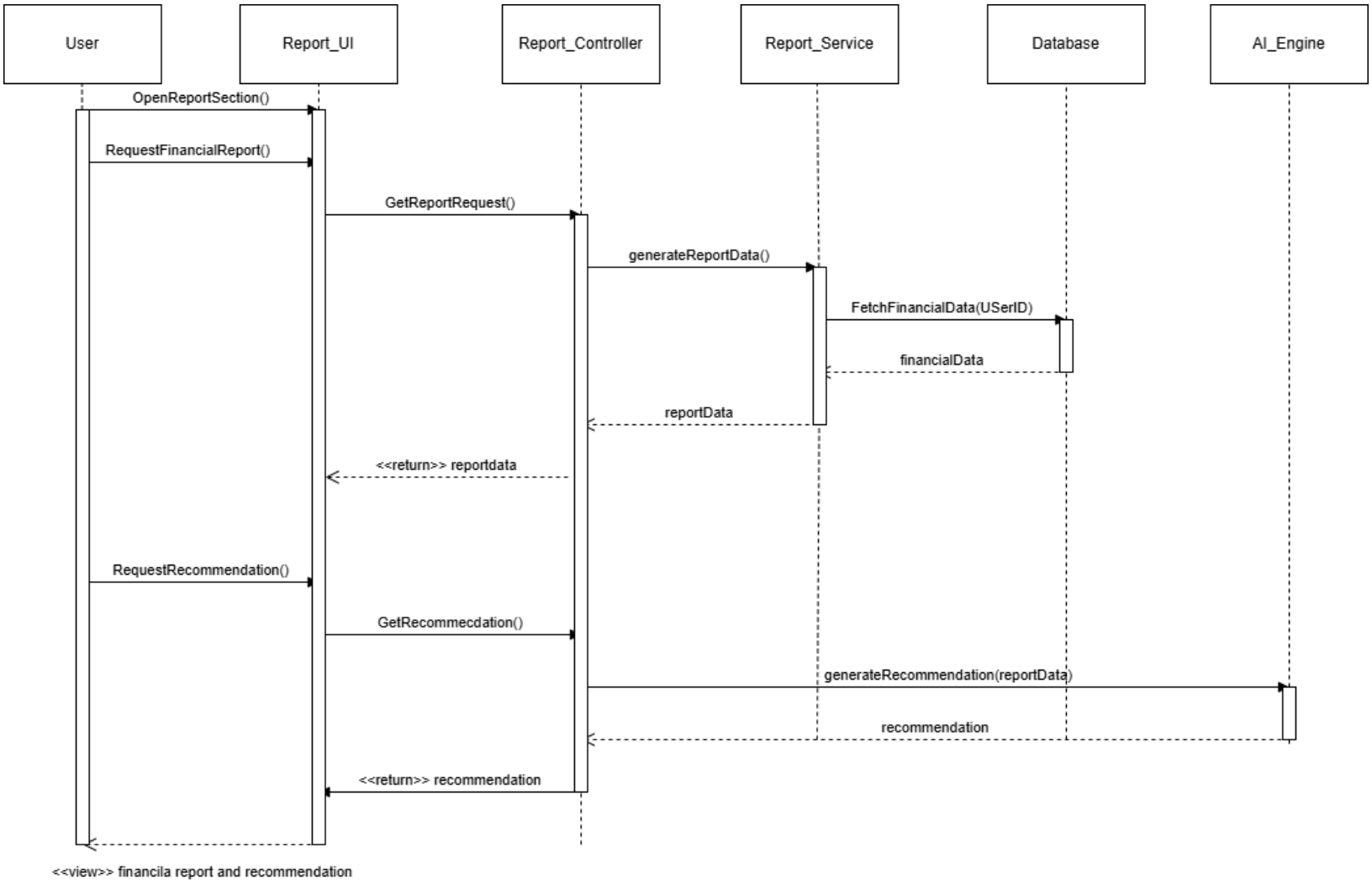
Sequence diagram 4



CS251: The Originals
Project: Personal Budgeting

Software Design Specification

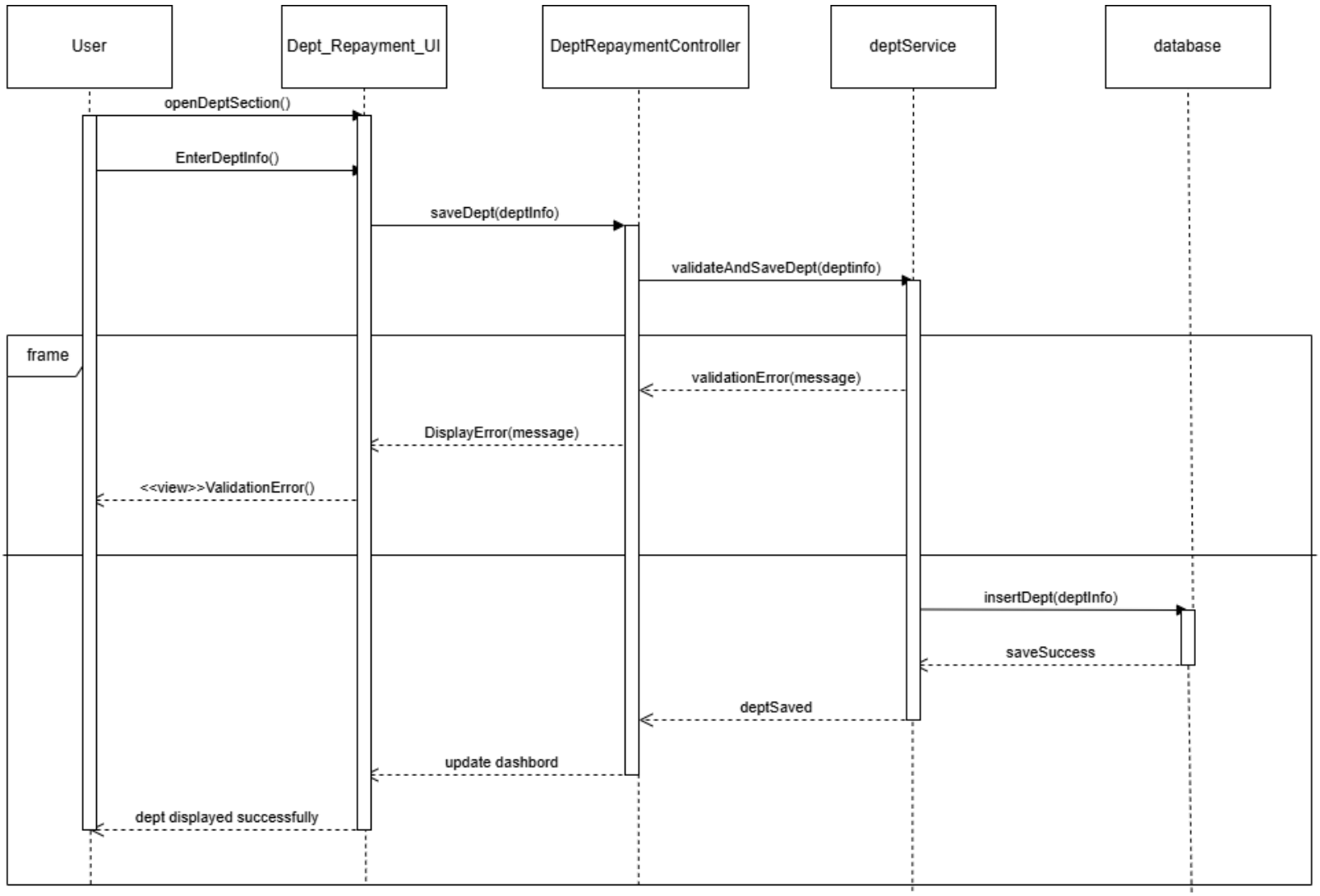
Sequence diagram 5



CS251: The Originals
Project: Personal Budgeting

Software Design Specification

Sequence diagram 6



Software Design Specification

Class - Sequence Usage Table

Sequence Diagram	Classes Used	All Methods Used
1. Expense Tracking	• User • ExpenseEntryUI • ExpenseController • ExpenseService • DataBase • ConfirmationUI	• OpenExpenseEntryUI() • EnterExpenseDetails(data) • submitExpense(data) • ShowInvalidData() • InvalidDataDisplayed • SuccessMessageDisplayed • validateData(data) • InvalidData() • validationSuccess() • SaveExpense(data) • ShowSuccessMessage() • SaveExpense(data) • SaveSuccess()
2. Budgeting & Analysis	• User • Budgeting_UI • Budget_Controller • Budget_Service • DataBase • Analysis_Report_UI	• OpenBudgetForm() • EnterBudgetDetails(BudgetData) • SubmitBudget(budgetDetails) • ShowValidationError() • validateBudget(BudgetDetails) • DisplayError(message) • validationError(message) • saveBudget(budgetData) • FetchExpenses() • generateAnalysis(analysisData) • saveBudget(budgetData) • saveSuccess() • FetchExpenses() • expenseData • displayAnalysis(analysisData) • showAnalysisResults()
3. AI-Powered Expense Prediction	• User • Prediction_UI • prediction_controller • Ai_prediction_service • DataBase	• OpenPredictionScreen() • requestPrediction() • dispalyPrediction(predictedAmount) • ShowPrediction(Amount) • submitPredictoinRequest() (note: small typo in the diagram, should be **submitPredictionRequest()**) • generatePrediction(userID) • fetchUserExpense(userID) • PredictionResult(PredictedAmount) • fetchUserExpense(userID) • ExpenseHistory

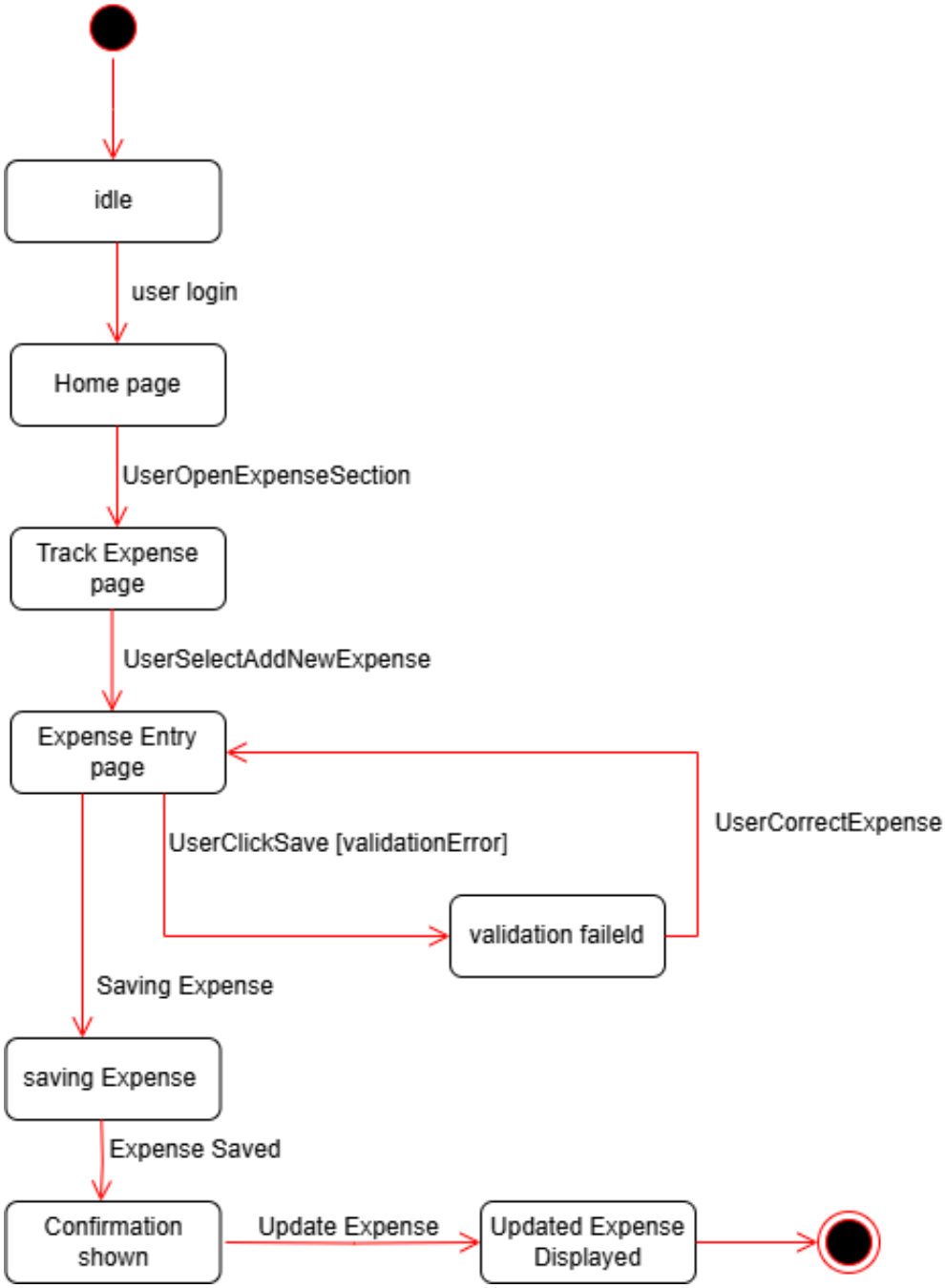
Software Design Specification

4. Transactions	• TransactionUI • TransactionController • TransactionService • Database • User	• EnterTransactionDetails(data) • SubmitTransaction(data) • ValidateAndSave()
5. Financial Reports and Insights	• ReportUI • ReportController • ReportService • Database • AiEngine • User	• RequestFinancialReport() • GetReportRequest() • GenerateReportData() • FetchFinancialData(UserId) • RequestRecommendation() • GetRecommendation()
6. Debt Repayment	• DeptRepaymentUI • DeptRepaymentController • DeptService • Database	• EnterDeptInfo() • SaveDept(DeptInfo) • ValidateAndSubmit(DeptInfo)

Software Design Specification

V. State Diagram

Expense Entry workflow



Software Design Specification

VI. SOLID Principles

1. Single Responsibility Principle (SRP)

- TransactionController handles only transactions.
- ExpenseValidator validates only expense data.
- ReportService is focused only on compiling reports.

2. Open/Closed Principle (OCP)

- Validator interface allows extension via ExpenseValidator, TransactionValidator without modifying the interface.
- ReportService can be extended to generate different kinds of reports without altering its core logic.

3. Liskov Substitution Principle (LSP)

- ExpenseValidator and TransactionValidator can be used anywhere Validator is expected.
- PredictionResult can be used wherever result data is processed, regardless of prediction method.



Software Design Specification

4. **Interface Segregation Principle (ISP)**

- Validator interface has only one method: validate(data).
- UserContext exposes only essential user-related methods like getCurrentUser() and isAuthenticated().

5. **Dependency Inversion Principle (DIP)**

- Controllers (TransactionController, BudgetController) depend on Validator abstraction.
- Services use domain models like ExpenseData, BudgetData through well-defined interfaces or parameters.

VII. Design Patterns

1. **MVC Design Pattern**

These classes represent the core data and business logic:

- User, Wallet, BudgetData, ExpenseData, DeptInfo, TransactionData, ReportData, PredictionResult
- Services like: BudgetService, ExpenseServices, TransactionService, AIpredictionService, DeptServices

These **model** and service classes(**Model**):

- Encapsulate state and logic (e.g., addCash, saveExpense, compileReport)
- Operate independently of the user interface



Software Design Specification

These are the user interface (UI) components(**View**):

- TransactionUI, ExpenseEntryUI, BudgetUI, ConfirmationUI, AnalysisReportUI, PredictionUI

Their job is to:

- Display forms or messages to the user
- Send user inputs to controllers

These classes handle user input and coordinate between the view and model:

- TransactionController, ExpenseController, BudgetController, PredictionController, ReportController, AccountController, DeptRepaymentController

Responsibilities:

- Receive data from UI (views)
- Validate input via validators (Strategy pattern)
- Use service classes to manipulate models
- Return responses or success messages to UI

2. Strategy Design Pattern

The Strategy Pattern is clearly shown in the use of validators.

Example:

- Validator is an interface with a method: validate(data: Object)
- Concrete implementations:
  - BudgetValidator, ExpenseValidator, TransactioValidator, DeptValidator

How it's Strategy:

- These validators encapsulate validation algorithms (strategies)
- They can be interchanged based on the data type



Software Design Specification

- Controllers like BudgetController, ExpenseController, etc., use a specific validation strategy

3. Observer Design Pattern

Although not explicitly shown in traditional event-subscription classes (like Observer/Observable), there are Observer-like behaviors implied in the interactions between models and services.

Potential Observers:

- The UI classes (views) could be observing model changes (e.g., showing updated predictions or analysis)
- Example:
    - PredictionUI calls PredictionController, which may receive updated PredictionResult
    - Similarly, AnalysisReportUI may show updated trends from AIpredictionService

If implemented fully, the Observer pattern would allow:

- Views (UI) to automatically update when models change
- Useful for real-time dashboards or prediction feedback

CS251: The Originals
Project: Personal Budgeting

Software Design Specification

Tools

- **Drawio and ArgoUML for all works and designs in the system**

Ownership Report

Owners	item
Abdallah Kareem Abdallah	Part of Class Diagram and SOLID Principles
Reyad Youssef Ebrahim	Part of Architecture Diagram and document audience and purpose
Hady Hassan Ahmed	Part of 6 Sequence Diagrams and state diagram and Design Patterns